

Dense embedding as a rerank signal — tested under paraphrase

Branch `dense_rerank` (from `stress_harness`). `RerankConfig.dense_weight`, `src/embed.py`.

The hypothesis

Every S6 rerank signal is exact-token: `span_weight * coverage, _facet_agreement, _category_match, _tail_match` all go to **zero** the moment the customer says "cowhide" instead of "leather" (see `docs/team/bm25.md` §5b and `docs/team/stress_harness.md`). A sentence-embedding cosine is the only signal that could score *meaning*, so it should be the fix for the paraphrase MRR collapse.

What was built

The `bge-small-en-v1.5` bi-encoder (384-dim, CLS pooling, ONNX, no torch) from `kwongweng_dense_retrieval`, wired as **one added term** in the rerank score:

```
total += dense_weight * minmax_over_head( cosine(turn_query_vec, candidate_vec) )
```

- one query vector per turn (`full_text()` by default; `dense_query="spans"` encodes just the disclosed spans);
- min-max normalised over the head, **not** divide-by-max — raw catalog cosines sit in a narrow $\sim[0.55, 0.80]$ band because every pool member is already the right category, so only the *differential* carries information;
- `dense_weight=0.0` (default) is byte-identical to before; missing `onnxruntime` / `tokenizers` / artifact -> silently 0.
- sweep rows `dense_rr_02/05/10/15` (weights 0.2–1.5) + `_spans` + `_rns`.

Results — `tools/stress_harness.py`, branch-default agent

run	router_on	dense_rr_10 (w=1.0)	Δ
official (cooperative)	0.91768	0.91537	-0.0023
paraphrase:heavy (full 200)	0.85577	0.85272	-0.0031
paraphrase:heavy + browse-gated (full 200)	0.80071	0.80048	-0.0002
paraphrase:heavy + browse-gated, --targets generic (21)	0.58621	0.60704	+0.0208

Generic subset, by weight: `dense_rr_05` +0.013, `dense_rr_10` +0.021 — monotone up. Hit@10 is unchanged in every row; the movement is entirely MRR (e.g. generic 0.348 -> 0.417). So the dense term reorders the *pool*, it does not change recall.

Reading it

The hypothesis is directionally right but the magnitude is small and situational. The bi-encoder cosine helps only where the exact-match signals have *completely* saturated — constraints that are all high-frequency **and** paraphrased (`--targets generic`, $\sim 10\%$ of sessions). On a realistic full set it is a **wash to slightly negative** (-0.003), because:

- 1 **The pool is category-homogeneous.** Retrieval already filtered to "belts", so bge cosines cluster tightly — the bi-encoder discriminates *category* strongly, *attributes* ("brown cowhide" vs "black canvas") weakly. Two independent embeddings can't focus on the attribute delta.

- 2 **Most paraphrased sessions keep enough verbatim tokens** (tok_cov 0.66 at heavy) that span coverage still fires and the dense term just dilutes it.
- 3 dense_weight high enough to rescue the generic tail (1.0) over-rides good exact matches elsewhere; a weight low enough to be safe on the full set barely moves the tail.

What's not claimed / next

- **Not shipped as a default.** dense_weight stays 0.0. It is a measured, documented option for the degenerate-tail case.
- **The real semantic-rerank lever is a cross-encoder** (bge-reranker-base, query+doc scored as a pair — the S6b stage on branch semantic-rerank-experiment, built and shelved against the *cooperative* simulator, never tested under paraphrase). It attends to the attribute delta a bi-encoder averages away. Cost is 10–50× per turn (~0.3–1.2 s for 20 candidates on CPU), so it needs a top-K gate. That is the experiment this result points to.
- **Loosening the exact-substring span match to token-set overlap** is the cheaper first move and is still untested (docs/team/stress_harness.md implications).

Reproduce

```
git checkout dense_rerank
pip install -r requirements.txt
python3 -m unittest discover -s tests -t .
python3 tools/stress_harness.py --customer paraphrase:heavy+browse-gated --configs router_on,dense_rr_05,dense_rr_06
python3 tools/stress_harness.py --customer paraphrase:heavy+browse-gated --targets generic --configs router_on,de
```

(dense configs are slow on CPU — one ONNX encode per turn, ~15–20 min per run.)